2

2.1

StarryOS `Thread` `ProcessData`

StarryOS `fork` `clone` `clone3` `execve` `exit` `waitpid`

pidfd

“ ”

clone exec `execveat` wait UID/GID ptrace pidfd

moss StarryOS

	StarryOS	moss
	`Thread` `ProcessData`	/ namespace
	`clone3` `execve`	clone ABI `execveat`
	`waitpid`	`waitid` pidfd
	pidfd	ptrace

2-1

2.2

StarryOS moss

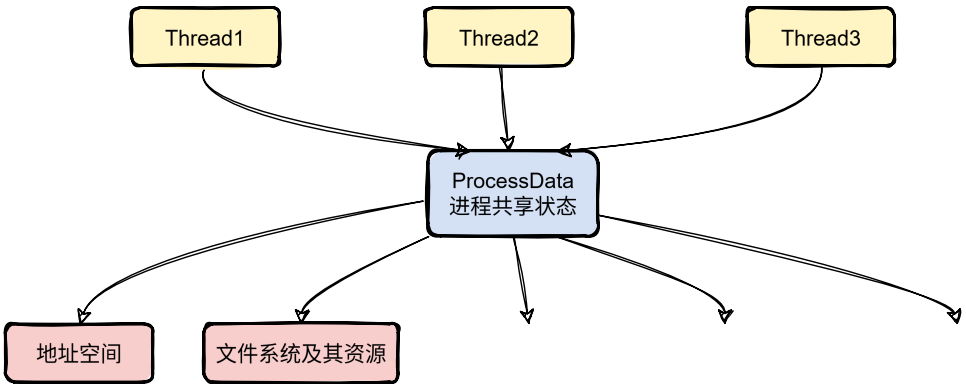
2.2.1

moss StarryOS

`Thread` `ProcessData` `ProcessData` clone

Thread

ProcessData clone exec exit



```rust
pub struct Thread {
    // 共享 程 源
    pub proc_data: Arc<ProcessData>,
    //
    pub signal: Arc<ThreadSignalManager>,
    pub exit: Arc<AtomicBool>,
}

pub struct ProcessData {
    //
    pub proc: Arc<Process>,
    //
    pub aspace: Arc<Mutex<AddrSpace>>,
    pub scope: RwLock<Scope>,
    pub wait_status: SpinNoIrq<Option<ProcessWaitStatus>>,
    // UID/GID 三元
    uid: AtomicU32, euid: AtomicU32, suid: AtomicU32,
    gid: AtomicU32, egid: AtomicU32, sgid: AtomicU32,
}
```

2-1

`Arc<ProcessData>`

`Thread` moss

2.3.2

StarryOS moss

`ProcessWaitStatus`

```rust
pub enum ProcessWaitStatus {
    // 被信号 停，但 程仍然存在
    Stopped(Signo),
    // 从停止状 恢复
    Continued(Signo),
}
```

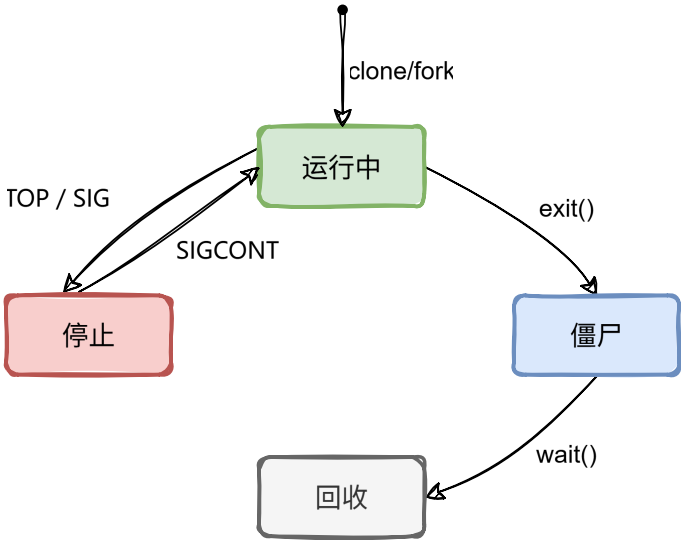
2-2

`Process` `ProcessData` 2-2

SIGSTOP/SIGCONT

exit wait wait

Stopped Continued



2-2

2.3.3

wait

——

Shell Ctrl+Z SIGSTOP fg

bg Shell “ ” “ ” wait

Shell WUNTRACED ——

WCONTINUED SIGCONT

PID ECHILD

WNOHANG	
WUNTRACED	
WCONTINUED	
WNOWAIT	

2-2

2-3 wait

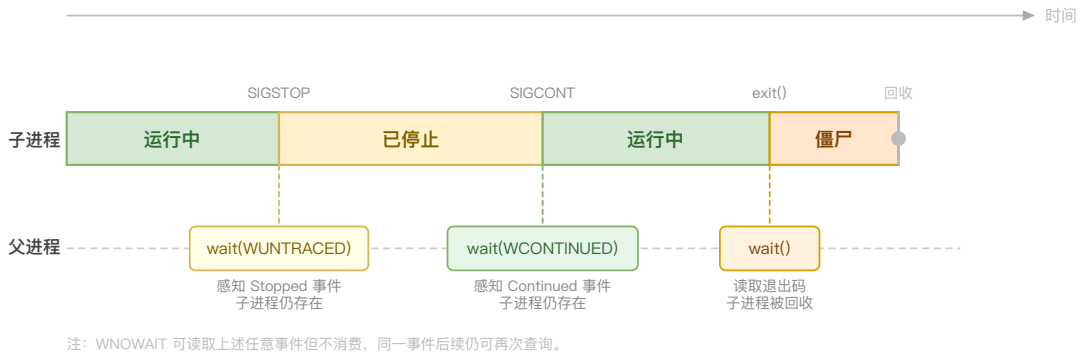
WNOHANG

WNOWAIT

waitpid wait4 moss waitid

siginfo_t moss

wait



2-3 wait

`waitid` pidfd pidfd PID

`WNOWAIT`

2.3.4

TID PID PGID SID moss

`TASK_TABLE`	TID	
`PROCESS_TABLE`	PID	
`PROCESS_GROUP_TABLE`	PGID	
`SESSION_TABLE`	SID	

2-3

moss

PGID SID

2.4

2.4.1 fork clone

`fork` moss x86_64 `sys_fork` `sys_clone`

`SIGCHLD` `fork` `ProcessData`

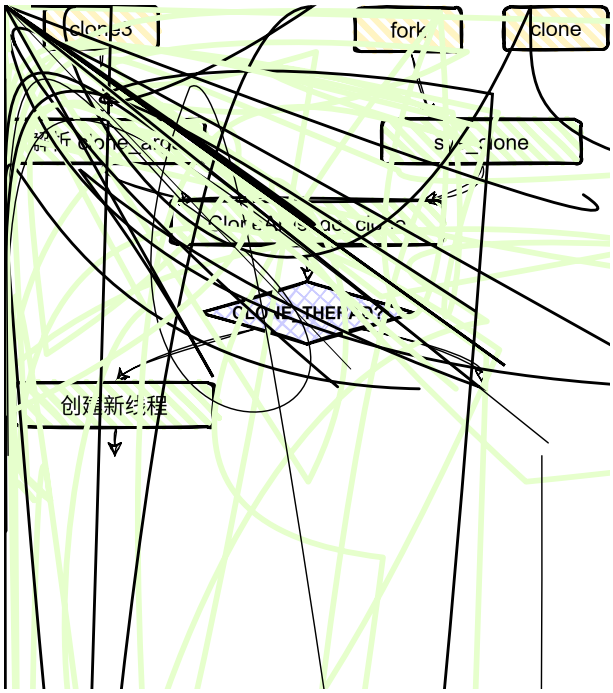
`clone` `clone3`

`CloneArgs` `CloneArgs::do_clone`

`CLONE_THREAD` `ProcessData` `Process::fork`

`ProcessData`

`CLONE_VM` `CLONE_FILES` `CLONE_FS` `CLONE_SIGHAND`


clone
fork
clone
CLONE_THEN
创建新线程

```
    return Err(AxError::InvalidInput);
}
```

2-3 clone

clone3 moss

2.4.3

2.3.3 2.3.3 wait

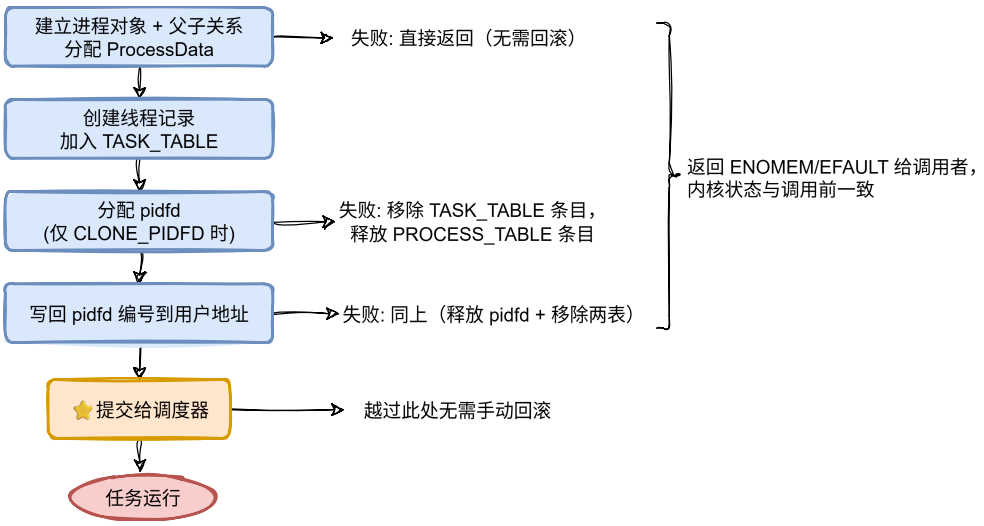
clone

PID

do_clone ProcessData

CLONE_PIDFD

2-5



2-5 do_clone

pidfd pidfd

PROCESS_TABLE TASK_TABLE

exit()

waitpid(-1, ...) PID PROCESS_TABLE

children

moss pidfd

d

PID

`MS_NOEXEC` `execveat`

2.5.2 execveat

`execve`

`execveat` `AT_EMPTY_PATH`

moss `execve` `execveat` `execve_common`

moss

```
// 挂  点禁止  行
if exec_loc.mount_flags() & MS_NOEXEC != 0 {
    return Err(AxError::PermissionDenied);
}
// 按有效 UID/GID
if !has_exec_permission(exec_loc, proc_data.euid(), proc_data.egid())? {
    return Err(AxError::PermissionDenied);
}
// 拒  正在被写入的可  行文件
if is_text_busy(exec_loc) {
    return Err(AxError::from(LinuxError::ETXTBSY));
}
```

2-4

2.6

2.6.1

robust futex `clear_child_tid`

futex

`ProcessData`

moss

ptrace `wait`

2.6.2 exit_group

`exit` `exit_group` moss

2.6.3

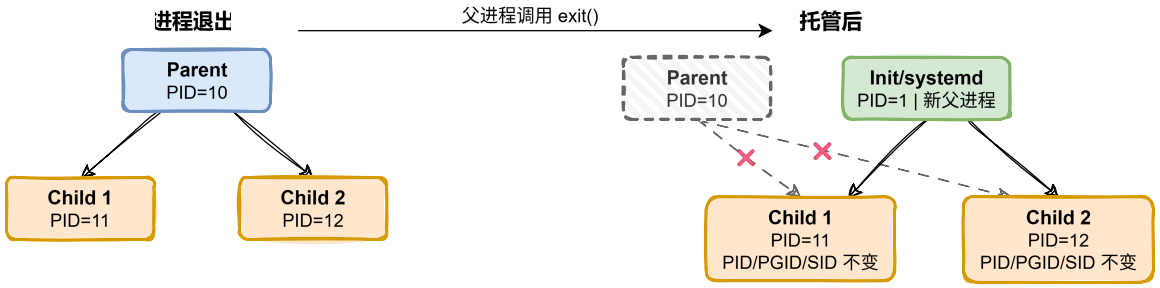
——

moss

StarryOS `children` init 2-6

PID PGID SID

init `wait`



2-6

2.7

2.7.1 PID PGID SID

PID PGID

SID PID PGID PID SID Shell

moss StarryOS `Process` PID `ProcessGroup`

`ProcessGroup` PGID `Session` `Session` SID

```
pub struct Process {
    pid: Pid,
    // 每个  程只属于一个  程
    group: SpinNoIrq<Arc<ProcessGroup>>,
}

pub struct ProcessGroup {
    pgid: Pid,
    //
    session: Arc<Session>,
    //
    processes: SpinNoIrq<WeakMap<Pid, Weak<Process>>>,
}
```

```rust
pub struct Session {
    sid: Pid,
    // 一个会  可以包含多个  程
    process_groups: SpinNoIrq<WeakMap<Pid, Weak<ProcessGroup>>>,
}
```

2-5

PID

——

`ProcessGroup::processes`

2.7.2

StarryOS `setpgid` `getpgid` PGID

PGID

moss PGID PGID PID

PGID

2.7.3

`setsid` moss `Session`

`ProcessGroup`

```rust
if let Some((session, group)) = proc.create_session() {
    // setsid 同    建会  和  程
    add_session_to_table(&session);
    add_process_group_to_table(&group);
    // SID 等于  用  程的 PID
    Ok(session.sid() as _)
}
```

2-6

2.8

2.8.1 UID/GID

UID

moss Linux real UID

effective UID saved UID GID

set-user-ID

effective UID

saved UID

StarryOS

moss `ProcessData` UID/GID

`setresuid` `setresgid`

```
let allowed = [old_ruid, old_euid, old_suid];
if old_euid != 0 {
    // 非特    程不能把三元  改成从未  有  的身份
    if !valid_id_change(ruid, &allowed)
        || !valid_id_change(euid, &allowed)
        || !valid_id_change(suid, &allowed)
    {
        return Err(eperm());
    }
}
// 分  更新三个字段，UID_GID_NO_CHANGE 表示不修改
if ruid != UID_GID_NO_CHANGE { proc_data.set_real_uid(ruid); }
if euid != UID_GID_NO_CHANGE { proc_data.set_euid(euid); }
if suid != UID_GID_NO_CHANGE { proc_data.set_suid(suid); }
```

2-7 UID

real effective saved

effective UID 0 effective UID/GID

`kill`

ptrace

capability root moss effective permitted inheritable bounding

2.8.2 ptrace

moss ptrace `TRACEME` `ATTACH` `DETACH` wait

attach

wait

`PTRACE_SYSCALL` `PTRACE_SINGLESTEP`

siginfo `EIO` GDB

2.9

moss StarryOS `Thread` `ProcessData`

real effective saved ID clone exec exit wait

moss `clone3` ABI `execveat`

ptrace vfork

exec

namespace ptrace StarryOS